

Data masking or data obfuscation is the process of modifying sensitive data in such a way that it is of no or little value to unauthorized intruders while still being usable by software or authorized personnel. Data masking can also be referred as anonymization, or tokenization, depending on different context.

The main reason to mask data is to protect information that is classified as personally identifiable information, or mission critical data. However, the data must remain usable for the purposes of undertaking valid test cycles. It must also look real and appear consistent. It is more common to have masking applied to data that is represented outside of a corporate production system. In other words, where data is needed for the purpose of application development, building program extensions and conducting various test cycles. It is common practice in enterprise computing to take data from the production systems to fill the data component, required for these non-production environments. However, this practice is not always restricted to non-production environments. In some organizations, data that appears on terminal screens to call center operators may have masking dynamically applied based on user security permissions (e.g. preventing call center operators from viewing credit card numbers in billing systems).

The primary concern from a corporate governance perspective is that personnel conducting work in these non-production environments are not always security cleared to operate with the information contained in the production data. This practice represents a security hole where data can be copied by unauthorized personnel, and security measures associated with standard production level controls can be easily bypassed. This represents an access point for a data security breach.

Background

Data involved in any data masking or obfuscation must remain meaningful at several levels:

The data must remain meaningful for the application logic. For example, if elements of addresses are to be obfuscated and city and suburbs are replaced with substitute cities or suburbs, then, if within the application there is a feature that validates postcode or post code lookup, that function must still be allowed to operate without error and operate as expected. The same is also true for credit-card algorithm validation checks and Social Security Number validations.

The data must undergo enough changes so that it is not obvious that the masked data is from a source of production data. For example, it may be common knowledge in an organisation that there are 10 senior managers all earning in excess of \$300k. If a test environment of the organisation's HR System also includes 10 identities in the same earning-bracket, then other information could be pieced together to reverse-engineer a real-life identity. Theoretically, if the data is obviously masked or obfuscated, then it would be reasonable for someone intending a data breach to assume that they could reverse engineer identity-data if they had some degree of knowledge of the identities in the production data-set. Accordingly, data obfuscation or masking of a data-set applies in such a manner as to ensure that identity and sensitive data records are protected - not just the individual data elements in discrete fields and tables.

The masked values may be required to be consistent across multiple databases within an organization when the databases each contain the specific data element being masked. Applications may initially access one database and later access another one to retrieve related information where the foreign key has been masked (e.g. a call center application first brings up data from a customer master database and, depending on the situation, subsequently accesses one of several other databases with very different financial products.) This requires that the masking applied is repeatable (the same input value to the masking algorithm always yields the same output value) but not able to be reverse engineered to get back to the original value. Additional constraints as mentioned in (1) above may also apply depending on the data

element(s) involved. Where different character sets are used across the databases that need to connect in this scenario, a scheme of converting the original values to a common representation will need to be applied, either by the masking algorithm itself or prior to invoking said algorithm.

Techniques

Substitution

Substitution is one of the most effective methods of applying data masking and being able to preserve the authentic look and feel of the data records.

It allows the masking to be performed in such a manner that another authentic-looking value can be substituted for the existing value. There are several data field types where this approach provides optimal benefit in disguising the overall data subset as to whether or not it is a masked data set. For example, if dealing with source data which contains customer records, real life surname or first name can be randomly substituted from a supplied or customised look up file. If the first pass of the substitution allows for applying a male first name to all first names, then the second pass would need to allow for applying a female first name to all first names where gender equals "F." Using this approach we could easily maintain the gender mix within the data structure, apply anonymity to the data records but also maintain a realistic looking database, which could not easily be identified as a database consisting of masked data.

This substitution method needs to be applied for many of the fields that are in database structures across the world, such as telephone numbers, zip codes and postcodes, as well as credit card numbers and other card type numbers like Social Security numbers and Medicare numbers where these numbers actually need to conform to a checksum test of the Luhn algorithm.

In most cases, the substitution files will need to be fairly extensive so having large substitution

datasets as well the ability to apply customized data substitution sets should be a key element of the evaluation criteria for any data masking solution.

Shuffling

The shuffling method is a very common form of data obfuscation. It is similar to the substitution method but it derives the substitution set from the same column of data that is being masked. In very simple terms, the data is randomly shuffled within the column. However, if used in isolation, anyone with any knowledge of the original data can then apply a "what if" scenario to the data set and then piece back together a real identity. The shuffling method is also open to being reversed if the shuffling algorithm can be deciphered.

Data shuffling overcomes reservations about using perturbed or modified confidential data because it retains all the desirable properties of perturbation while performing better than other masking techniques in both data utility and disclosure risk.

Shuffling, however, has some real strengths in certain areas. If for instance, the end of year figures for financial information in a test data base, one can mask the names of the suppliers and then shuffle the value of the accounts throughout the masked database. It is highly unlikely that anyone, even someone with intimate knowledge of the original data could derive a true data record back to its original values.

Number and date variance

The numeric variance method is very useful for applying to financial and date driven information fields. Effectively, a method utilising this manner of masking can still leave a meaningful range in a financial data set such as payroll. If the variance applied is around +/- 10% then it is still a very meaningful data set in terms of the ranges of salaries that are paid to the recipients.

The same also applies to the date information. If the overall data set needs to retain

demographic and actuarial data integrity, then applying a random numeric variance of +/- 120 days to date fields would preserve the date distribution, but it would still prevent traceability back to a known entity based on their known actual date or birth or a known date value for whatever record is being masked.

Encryption

Encryption is often the most complex approach to solving the data masking problem. The encryption algorithm often requires that a "key" be applied to view the data based on user rights. This often sounds like the best solution, but in practice the key may then be given out to personnel without the proper rights to view the data. This then defeats the purpose of the masking exercise. Old databases may then get copied with the original credentials of the supplied key and the same uncontrolled problem lives on.

Recently, the problem of encrypting data while preserving the properties of the entities got recognition and a newly acquired interest among the vendors and academia. New challenge gave birth to algorithms performing format-preserving encryption. These are based on the accepted Advanced Encryption Standard (AES) algorithmic mode recognized by NIST.

Nulling out or deletion

Sometimes a very simplistic approach to masking is adopted through applying a null value to a particular field. The null value approach is really only useful to prevent visibility of the data element.

In almost all cases, it lessens the degree of data integrity that is maintained in the masked data set. It is not a realistic value and will then fail any application logic validation that may have been applied in the front end software that is in the system under test. It also highlights to anyone that wishes to reverse engineer any of the identity data that data masking has been applied to some degree on the data set.

Masking out

Character scrambling or masking out of certain fields is also another simplistic yet very effective method of preventing sensitive information to be viewed. It is really an extension of the previous method of nulling out, but there is a greater emphasis on keeping the data real and not fully masked all together.

This is commonly applied to credit card data in production systems. For instance, an operator at a call centre might bill an item to a customer's credit card. They then quote a billing reference to the card with the last 4 digits of XXXX XXXX xxxx 6789. As an operator they can only see the last 4 digits of the card number, but once the billing system passes the customer's details for charging, the full number is revealed to the payment gateway systems. This system is not very effective for test systems, but it is very useful for the billing scenario detailed above. It is also commonly known as a dynamic data masking method.

Additional complex rules

Additional rules can also be factored into any masking solution regardless of how the masking methods are constructed. Product agnostic white papers are a good source of information for exploring some of the more common complex requirements for enterprise masking solutions, which include row internal synchronization rules, table internal synchronization rules and table to Table Synchronization Rules.

Different types

Data masking is tightly coupled with building test data. Two major types of data masking are static and on-the-fly data masking.

Static data masking

Static data masking is usually performed on the golden copy of the database, but can also be applied to values in other sources, including files. In DB environments, production database administrators will typically load table backups to a separate environment, reduce the dataset to a subset that holds the data necessary for a particular round of testing (a technique called "subsetting"), apply data masking rules while data is in stasis, apply necessary code changes from source control, and/or and push data to desired environment.

Deterministic data masking

Deterministic masking is the process of replacing a value in a column with the same value whether in the same row, the same table, the same database/schema and between instances/servers/database types. Example: A database has multiple tables, each with a column that has first names. With deterministic masking the first name will always be replaced with the same value – "Lynne" will always become "Denise" – wherever "Lynne" may be in the database.

Statistical data obfuscation

There are also alternatives to the static data masking that rely on stochastic perturbations of the data that preserve some of the statistical properties of the original data. Examples of statistical data obfuscation methods include differential privacy and the DataSifter method.

On-the-fly data masking

On-the-fly data masking happens in the process of transferring data from environment to environment without data touching the disk on its way. The same technique is applied to "Dynamic Data Masking" but one record at a time. This type of data masking is most useful for environments that do continuous deployments as well as for heavily integrated applications.

Organizations that employ continuous deployment or continuous delivery practices do not have the time necessary to create a backup and load it to the golden copy of the database. Thus, continuously sending smaller subsets (deltas) of masked testing data from production is important. In heavily integrated applications, developers get feeds from other production systems at the very onset of development and masking of these feeds is either overlooked and not budgeted until later, making organizations non-compliant. Having on-the-fly data masking in place becomes essential.

Dynamic data masking

Dynamic data masking is similar to on-the-fly data masking, but it differs in the sense that on-the-fly data masking is about copying data from one source to another source so that the latter can be shared. Dynamic data masking happens at runtime, dynamically, and on-demand so that there doesn't need to be a second data source where to store the masked data dynamically.

Dynamic data masking enables several scenarios, many of which revolve around strict privacy regulations e.g. the Singapore Monetary Authority or the Privacy regulations in Europe.

Dynamic data masking is attribute-based and policy-driven. Policies include:

Doctors can view the medical records of patients they are assigned to (data filtering)

Doctors cannot view the SSN field inside a medical record (data masking).

Dynamic data masking can also be used to encrypt or decrypt values on the fly especially when using format-preserving encryption.

Several standards have emerged in recent years to implement dynamic data filtering and masking. For instance, XACML policies can be used to mask data inside databases.

There are six possible technologies to apply Dynamic data masking:

In the database: Database receives the SQL and applies rewrite to returned masked result set. Applicable for developers and database administrators, but not for applications (because connection pools, application caching and data-bus hide the application user identity from the database and can also cause application data corruption).

Network proxy between the application and the database: Captures the SQL and applies rewrite on the select request. Applicable for developers and database administrators with simple 'select' requests but not for stored procedures (which the proxy only identifies the exec.) and applications (because connection pools, application caching and data-bus hide the application user identity from the database and can also cause application data corruption).

Database proxy: is a variation of network proxy. Database proxy is deployed usually between applications/users and the database. Applications and users are connecting to the database through database security proxy. There are no changes to the way applications and users are connecting to the database. There is also no need of an agent to be installed on the database server. The sql queries are rewritten, but when implemented, this type of dynamic data masking also supported within store procedures and database functions.

Network proxy between the end-user and the application: identifying text strings and replacing them. This method is not applicable for complex applications as it will easily cause corruption when the real-time string replacement is unintentionally applied.

Code changes in the applications & XACML: code changes are usually hard to perform, impossible to maintain and not applicable for packaged applications.

Within the application run-time: By instrumenting the application run-time, policies are defined to rewrite the result set returned from the data sources, while having full visibility to the application user. This method is the only applicable way to dynamically mask complex applications as it enables control to the data request, data result and user result.

Supported by a browser plugin: In the case of SaaS or local web applications, browser add-ons can be configured to mask data fields corresponding to precise CSS Selectors. This can either

be accomplished by marking sensitive fields in the application, for example by a HTML class or by finding the right selectors that identify the fields to be obfuscated or masked.

Data masking and the cloud

In latest years, organizations develop their new applications in the cloud more and more often, regardless of whether final applications will be hosted in the cloud or on- premises. The cloud solutions as of now allow organizations to use infrastructure as a service, platform as a service, and software as a service. There are various modes of creating test data and moving it from on-premises databases to the cloud, or between different environments within the cloud. Dynamic Data Masking becomes even more critical in cloud when customers need to protecting PII data while relying on cloud providers to administer their databases.

Data masking invariably becomes the part of these processes in the systems development life cycle (SDLC) as the development environments' service-level agreements (SLAs) are usually not as stringent as the production environments' SLAs regardless of whether application is hosted in the cloud or on-premises.

See also

Masking and unmasking by intelligence agencies

== References ==